

LG Soft — C++ Interview Questions

Transcribed from student-reported question sheets

Quick Reference — Key Concepts

STL container not supporting insertion in the middle (in constant time): `priority_queue` does not support this. (Random access, push/pop from both ends, and sorting are also not directly supported by every STL container — check the specific container in question.)

Question / Topic	Answer
<code>upper_bound()</code>	First element greater than key
<code>lower_bound()</code>	First element $\geq$ key
<code>priority_queue top()</code>	$O(1)$
<code>priority_queue push()</code>	$O(\log n)$
<code>priority_queue pop()</code>	$O(\log n)$
<code>priority_queue</code> constructor from range	$O(n)$
<code>vector push_back()</code>	Amortized $O(1)$
<code>sort()</code>	$O(n \log n)$
static variable	Shared among all objects
virtual function	Runtime polymorphism
friend function	Can access private members
constexpr	Compile-time evaluation
<code>[[nodiscard]]</code>	Compiler warning if return value ignored
<code>upper_bound(30) in {10,20,30,40,50}</code>	40
Macro <code>#define Square(x) x*x</code> with <code>Square(x+2)</code>	Wrong result due to missing parentheses

Question 1

What happens with the following code?

```
class Engine {
public:
    [[nodiscard("You should check if the engine starts properly!")]]
    bool start() {
        return false;
    }
};

int main() {
    Engine e;
    e.start();
    return 0;
}
```

```
}
```

- a) Compiler warning: "You should check if the engine starts properly!"
- b) Runtime warning: "You should check if the engine starts properly!"
- c) Compilation error since "nodiscard" is not a keyword or function name
- d) Program runs fine without any warning or error

Question 2

What is the time complexity of constructing pq2 below?

```
priority_queue<int> pq1;  
vector<int> v = {9, 8, 6, 10, 4, 2};  
priority_queue<int> pq2(v.begin(), v.end());
```

- a) $O(N)$ b) $O(N \log N)$ c) $O(N^2)$ d) $O(1)$

Question 3

What is the time complexity to access the top element of a heap in `std::priority_queue`?

- a) $O(1)$ b) $O(\log n)$ c) $O(n)$ d) $O(n^2)$

Question 4

```
class Container {  
    int capacity;  
public:  
    Container(int cap) { capacity = cap; }  
    friend void show();  
};  
void Container::show() {  
    Container container(10);  
    cout << "Capacity of container is " << container.capacity << endl;  
}  
int main() { show(); }
```

Question 5

```
class Animal {  
public:  
    virtual void speak() { std::cout << "Default animal sound" << std::endl; }  
};  
class Dog : public Animal {  
public:  
    void speak(int x) { std::cout << "Dog barks" << std::endl; }  
};  
int main() {  
    Animal *animal = new Dog();  
    animal->speak();  
    return 0;  
}
```

Question 6

```
int x = -1;  
char *ptr = new char[256];  
try {  
    if (x < 0) throw x;  
    if (ptr == NULL) throw "ptr is NULL";  
} catch (...) {
```

```
    cout << "Exception occurred, exiting" << endl;
}
```

Question 7

Which operation is NOT supported directly by C++ STL deque?

- a) *Random access using []*
- b) *Push and pop from both ends*
- c) *Sorting using sort()*
- d) *Insertion at the middle in constant time*

Question 8

```
vector<int> v1;
for (char i = 1; i <= 3; i++) v1.push_back(i);
cout << v1.size() << " " << v1.capacity();

vector<int> v2(4);
for (char i = 1; i <= 3; i++) v2[i-1] = i;
cout << v2.size() << " " << v2.capacity();
```

Question 9

Regarding consteval in C++:

- a) *Keyword does not exist*
- b) *It must always be evaluated at compile time*
- c) *It can be evaluated at runtime or compile time*
- d) *It enforces constant-ness of variables*

Question 10

```
#define Square(x) x*x
int main() {
    int x = 0;
    cout << Square(x+2);
}
```

Question 11

A priority_queue of pair<int,string> (min-heap via greater<>) is passed by value to printEvents(pq, currentTime), which pops and prints events matching currentTime inside a while(1) loop:

```
void printEvents(priority_queue<pair<int,string>, vector<pair<int,string>>,
                greater<pair<int,string>>> pq, int currentTime) {
    while (1) {
        // suppose currentTime is updated externally so it always reflects the
        current time
        if (pq.top().first == currentTime) {
            cout << "Event: " << pq.top().second
                << " has been started at " << pq.top().first << "\n";
            pq.pop();
        }
    }
}
```

- a) *Runs fine and will do its job of printing current events*

b) Program may crash at runtime

c) Passing priority_queue to the function is not correct

Question 12

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    vector<int> v = {10, 20, 30, 40, 50};
    cout << *upper_bound(v.begin(), v.end(), 30);
    return 0;
}
```

Question 13

```
int x = 10;
int &ref = x;
ref = 20;
cout << x << " ";
x = 30;
cout << ref;
```

Question 14

```
template <class T>
class A {
public:
    T func(T a, T b) { return (a * 1.0) / b; }
};
int main() {
    A<int> a1;
    cout << a1.func(3, 2) << " " << a1.func(3.0, 2.0);
}
```

Question 15

Note: the source sheet writes the pointer type as "Text" in main() but the class is defined as "Test", and "Test" has no default constructor.

```
int main() {
    Text *t = new Text();
    t->demoPrint();
    t->score = 10;
    t->printScore();
}

class Test {
public:
    int score;
    Test(int num) { score = num; }
    void demoPrint() { cout << "TEST of LG SOFT ENDCAM" << endl; }
    void printScore() { cout << "Score is: " << score << endl; }
};
```

Question 16

```
int cin;
cin >> cin;
cout << "cin: " << cin;
return 0;
```

Question 17

```
class Test {
public:
    static int count;
    Test& fun() {
        count++;
        cout << this->count << " ";
        return *this;
    }
};
int Test::count = 0;

int main() {
    Test t1;
    t1.fun().fun().fun().fun();
    Test t2;
    t2.fun();
}
```

Question 18

```
class Solution {
public:
    bool comp(int a, int b) { return a < b; }
    void printInDecreasing(vector<int>& nums) {
        sort(nums.begin(), nums.end(), comp);
        for (auto num : nums) cout << num << " ";
    }
};

int main() {
    Solution sol;
    vector<int> nums = {1, 2, 3, 4, 5};
    sol.printInDecreasing(nums);
}
```